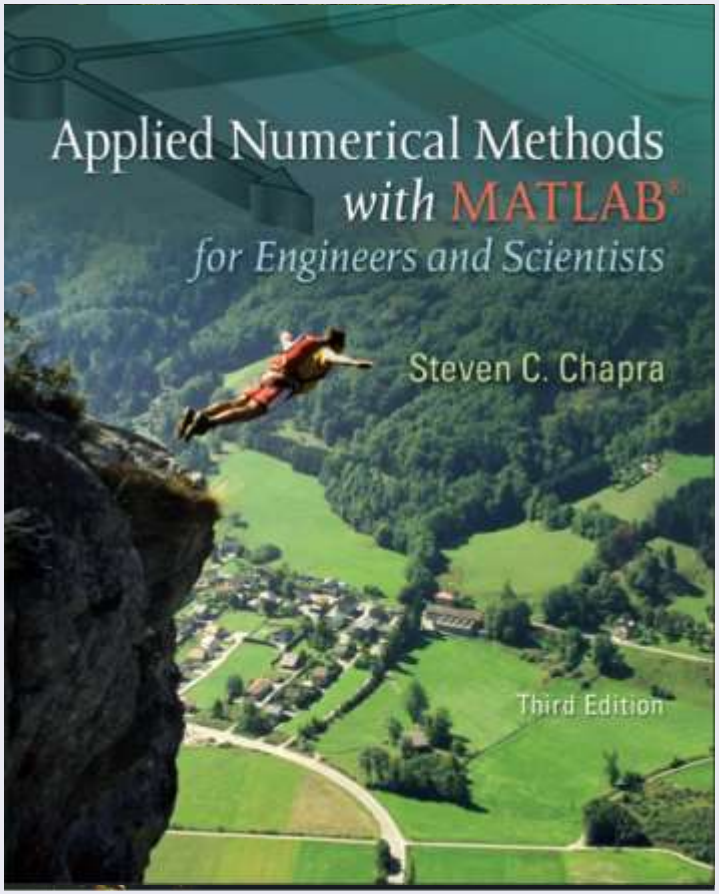


Part 1
Chapter 2

MATLAB Fundamentals

PowerPoints organized by Dr. Michael R. Gustafson II, Duke University and Prof. Steve Chapra, Tufts University

All images copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

Chapter Objectives

- Learning how real and complex numbers are assigned to variables.
- Learning how vectors and matrices are assigned values using simple assignment, the color operator, and the `linspace` and `logspace` functions.
- Understanding the priority rules for constructing mathematical expressions.
- Gaining a general understanding of built-in functions and how you can learn more about them with MATLAB's Help facilities.
- Learning how to use vectors to create a simple line plot based on an equation.

The MATLAB Environment

- MATLAB uses three primary windows-
  - Command window - used to enter commands and data
  - Graphics window(s) - used to display plots and graphics
  - Edit window - used to create and edit M-files (programs)
- Depending on your computer platform and the version of MATLAB used, these windows may have different looks and feels.

Calculator Mode

- The MATLAB command window can be used as a calculator where you can type in commands line by line. Whenever a calculation is performed, MATLAB will assign the result to the built-in variable `ans`
- Example:

```
>> 55 - 16
ans =
     39
```

MATLAB Variables

- While using the `ans` variable may be useful for performing quick calculations, its transient nature makes it less useful for programming.
- MATLAB allows you to assign values to variable names. This results in the storage of values to memory locations corresponding to the variable name.
- MATLAB can store individual values as well as arrays; it can store numerical data and text (which is actually stored numerically as well).
- MATLAB does not require that you pre-initialize a variable; if it does not exist, MATLAB will create it for you.

Scalars

- To assign a single value to a variable, simply type the variable name, the = sign, and the value:

```
>> a = 4
a =
   4
```

- Note that variable names must start with a letter, though they can contain letters, numbers, and the underscore (_) symbol

Scalars (cont)

- You can tell MATLAB not to report the result of a calculation by appending the semi-solon (;) to the end of a line. The calculation is still performed.
- You can ask MATLAB to report the value stored in a variable by typing its name:

```
>> a
a =
   4
```

Scalars (cont)

- You can use the complex variable `i` (or `j`) to represent the unit imaginary number.
- You can tell MATLAB to report the values back using several different formats using the format command. Note that the values are still *stored* the same way, they are just displayed on the screen differently. Some examples are:
  - `short` - scaled fixed-point format with 5 digits
  - `long` - scaled fixed-point format with 15 digits for double and 7 digits for single
  - `short eng` - engineering format with at least 5 digits and a power that is a multiple of 3 (useful for SI prefixes)

Format Examples

- ```
  >> format short; pi
  ans =
      3.1416
  >> format long; pi
  ans =
     3.14159265358979
  >> format short eng; pi
  ans =
       3.1416e+000
  >> pi*10000
  ans =
      31.4159e+003
  ```
- Note - the format remains the same unless another `format` command is issued.
  ```

Arrays, Vectors, and Matrices

- MATLAB can automatically handle rectangular arrays of data - one-dimensional arrays are called *vectors* and two-dimensional arrays are called *matrices*.
- Arrays are set off using square brackets [and] in MATLAB
- Entries within a row are separated by spaces or commas
- Rows are separated by semicolons

Array Examples

```
>> a = [1 2 3 4 5 ]
a =
     1     2     3     4     5
>> b = [2;4;6;8;10]
b =
     2
     4
     6
     8
    10
```

- Note 1 - MATLAB does not *display* the brackets
- Note 2 - if you are using a monospaced font, such as Courier, the displayed values should line up properly

Matrices

- A 2-D array, or matrix, of data is entered row by row, with spaces (or commas) separating entries within the row and semicolons separating the rows:

```
>> A = [1 2 3; 4 5 6; 7 8 9]

A =

     1     2     3
     4     5     6
     7     8     9
```

Useful Array Commands

- The transpose operator (apostrophe) can be used to flip an array over its own diagonal. For example, if `b` is a row vector, `b'` is a column vector containing the complex conjugate of `b`.
- The command window will allow you to separate rows by hitting the Enter key - script files and functions will allow you to put rows on new lines as well.
- The `who` command will report back used variable names; `whos` will also give you the size, memory, and data types for the arrays.

Accessing Array Entries

- Individual entries within a array can be both read and set using either the *index* of the location in the array or the row and column.
- The index value starts with 1 for the entry in the top left corner of an array and increases down a column - the following shows the indices for a 4 row, 3 column matrix:

```
1       5       9
2       6      10
3       7      11
4       8      12
```

Accessing Array Entries (cont)

- Assuming some matrix C:

```
C =
   2      4      9
   3      3     16
   3      0      8
  10     13     17
```

- `C(2)` would report 3
- `C(4)` would report 10
- `C(13)` would report an error!
- Entries can also be access using the row and column:
- `C(2,1)` would report 3
- `C(3,2)` would report 0
- `C(5,1)` would report an error!

Array Creation - Built In

- There are several built-in functions to create arrays:
  - `zeros(r,c)` will create an `r` row by `c` column matrix of zeros
  - `zeros(n)` will create an `n` by `n` matrix of zeros
  - `ones(r,c)` will create an `r` row by `c` column matrix of ones
  - `ones(n)` will create an `n` by `n` matrix one ones
- `help elmat` has, among other things, a list of the elementary matrices

Array Creation - Colon Operator

- The colon operator : is useful in several contexts. It can be used to create a linearly spaced array of points using the notation
  `start:diffval:limit`
  where `start` is the first value in the array, `diffval` is the difference between successive values in the array, and `limit` is the *boundary* for the last value (though not necessarily the last value).

```
>>1:0.6:3
ans =
    1.0000   1.6000   2.2000   2.8000
```

Colon Operator - Notes

- If `diffval` is omitted, the default value is 1:

```
>>3:6
ans =
     3     4     5     6
```

- To create a decreasing series, `diffval` must be negative:

```
>> 5:-1.2:2
ans =
    5.0000    3.8000    2.6000
```

- If `start+diffval>limit` for an increasing series or `start+diffval<limit` for a decreasing series, an empty matrix is returned:

```
>>5:2
ans =
   Empty matrix: 1-by-0
```

- To create a column, transpose the output of the colon operator, not the limit value; that is, `(3:6)'` not `3:6'`

Array Creation - `linspace`

- To create a row vector with a specific number of linearly spaced points between two numbers, use the `linspace` command.
- `linspace(x1, x2, n)` will create a linearly spaced array of `n` points between `x1` and `x2`

```
>>linspace(0, 1, 6)
ans =
  0  0.2000  0.4000  0.6000  0.8000  1.0000
```

- If `n` is omitted, 100 points are created.
- To generate a column, transpose the output of the `linspace` command.

Array Creation - `logspace`

- To create a row vector with a specific number of logarithmically spaced points between two numbers, use the `logspace` command.
- `logspace(x1, x2, n)` will create a logarithmically spaced array of `n` points between 10^{x1} and 10^{x2}

```
>>logspace(-1, 2, 4)
ans =
   0.1000    1.0000   10.0000  100.0000
```

- If `n` is omitted, 100 points are created.
- To generate a column, transpose the output of the `logspace` command.

Character Strings & Ellipsis

- Alphanumeric constants are enclosed by apostrophes (')

```
>> f = 'Miles ';
>> s = 'Davis'
```

- Concatenation: pasting together of strings

```
>> x = [f s]
x =
Miles Davis
```

- Ellipsis (...): Used to continue long lines

```
>> a = [1 2 3 4 5 ...
 6 7 8]
 a =
     1     2     3     4     5     6     7     8
```

- You cannot use an ellipsis within single quotes to continue a string. But you can piece together shorter strings with ellipsis

```
>> quote = ['Any fool can make a rule,' ...
 ' and any fool will mind it']
 quote =
 Any fool can make a rule, and any fool will mind it
```

Mathematical Operations

- Mathematical operations in MATLAB can be performed on both scalars and arrays.
- The common operators, in order of priority, are:

^	Exponentiation	4^2 = 8
-	Negation (unary operation)	-8 = -8
* /	Multiplication and Division	2*pi = 6.2832 pi/4 = 0.7854
\	Left Division	6\2 = 0.3333
+ -	Addition and Subtraction	3+5 = 8 3-5 = -2

Order of Operations

- The order of operations is set first by parentheses, then by the default order given above:
  - `y = -4 ^ 2` gives `y = -16`
    since the exponentiation happens first due to its higher default priority, but
  - `y = (-4) ^ 2` gives `y = 16`
    since the negation operation on the 4 takes place first

Complex Numbers

- All the operations above can be used with complex quantities (i.e. values containing an imaginary part entered using `i` or `j` and displayed using `i`)

```
>> x = 2+i*4; (or 2+4i, or 2+j*4, or 2+4j)
>> y = 16;
>> 3 * x
ans =
   6.0000 +12.0000i
>> x+y
ans =
  18.0000 + 4.0000i
>> x'
ans =
    2.0000 - 4.0000i
```

Vector-Matrix Calculations

- MATLAB can also perform operations on vectors and matrices.
- The * operator for matrices is defined as the *outer product* or what is commonly called “matrix multiplication.”
  - The number of columns of the first matrix must match the number of rows in the second matrix.
  - The size of the result will have as many rows as the first matrix and as many columns as the second matrix.
  - The exception to this is multiplication by a 1x1 matrix, which is actually an array operation.
- The ^ operator for matrices results in the matrix being matrix-multiplied by itself a specified number of times.
  - Note - in this case, the matrix must be square!

Element-by-Element Calculations

- At times, you will want to carry out calculations item by item in a matrix or vector. The MATLAB manual calls these *array operations*. They are also often referred to as *element-by-element* operations.
- MATLAB defines .* and ./ (note the dots) as the array multiplication and array division operators.
  - For array operations, both matrices must be the same size *or* one of the matrices must be 1x1
- Array exponentiation (raising each element to a corresponding power in another matrix) is performed with .^
  - Again, for array operations, both matrices must be the same size *or* one of the matrices must be 1x1

Built-In Functions

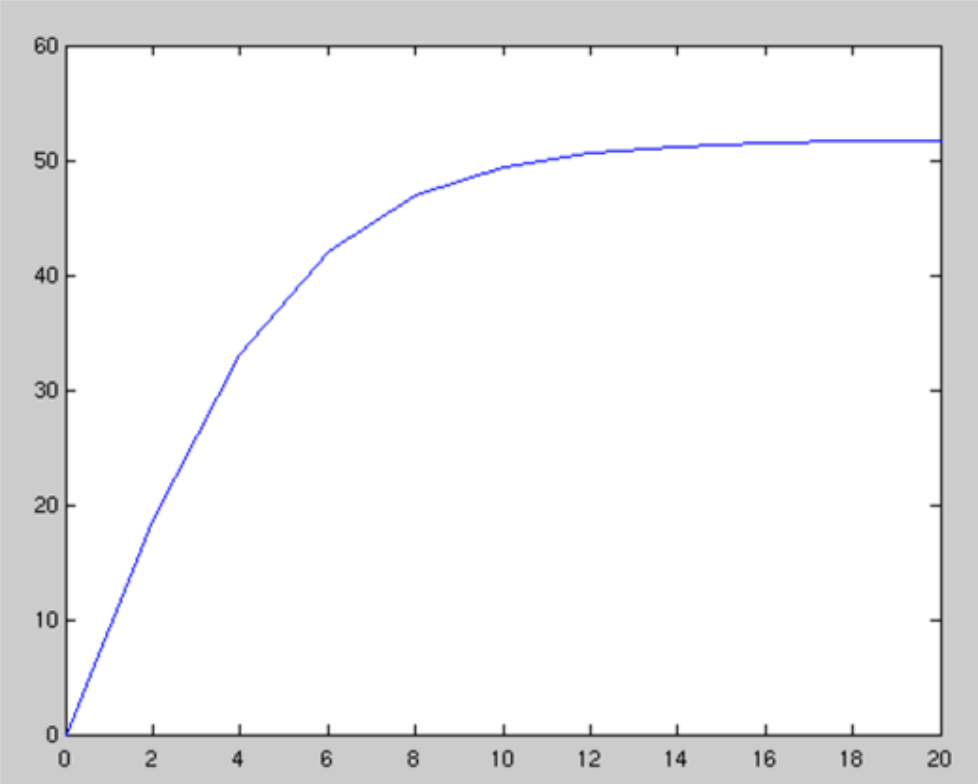
- There are several built-in functions you can use to create and manipulate data.
- The built-in help function can give you information about both what exists and how those functions are used:
  - `help elmat` will list the elementary matrix creation and manipulation functions, including functions to get information about matrices.
  - `help elfun` will list the elementary math functions, including trig, exponential, complex, rounding, and remainder functions.
- The built-in `lookfor` command will search help files for occurrences of text and can be useful if you know a function's purpose but not its name

Graphics

- MATLAB has a powerful suite of built-in graphics functions.
- Two of the primary functions are `plot` (for plotting 2-D data) and `plot3` (for plotting 3-D data).
- In addition to the plotting commands, MATLAB allows you to label and annotate your graphs using the `title`, `xlabel`, `ylabel`, and `legend` commands.

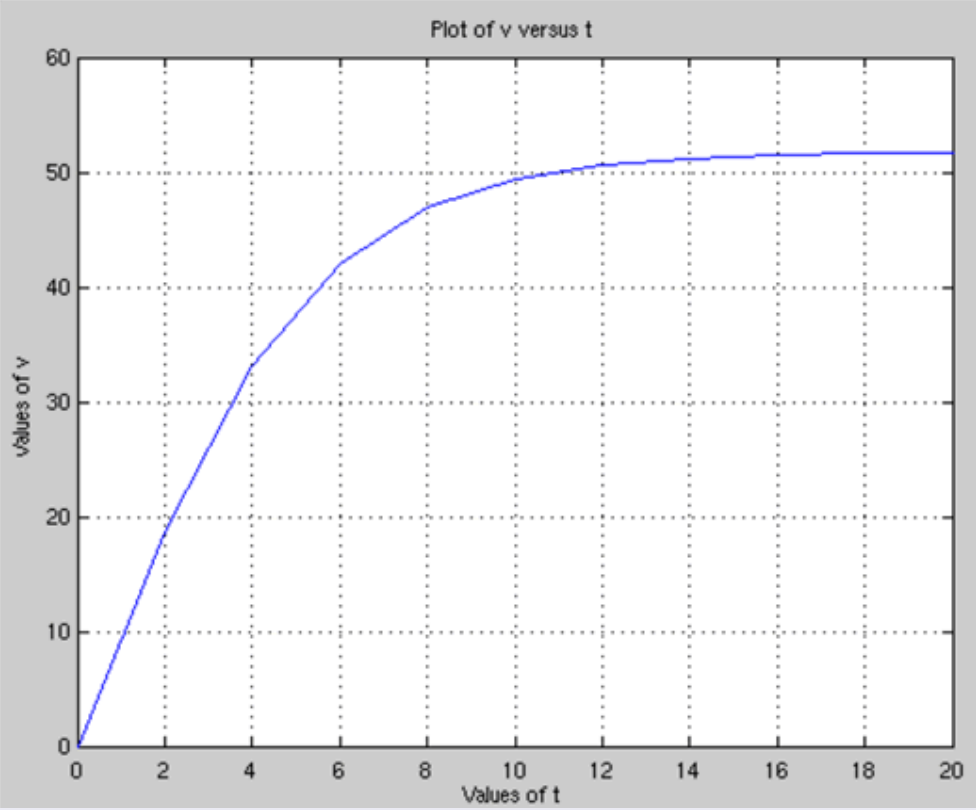
Plotting Example

```
t = [0:2:20]';
g = 9.81; m = 68.1; cd = 0.25;
v = sqrt(g*m/cd) * tanh(sqrt(g*cd/m)*t);
plot(t, v)
```



Plotting Annotation Example

```
title('Plot of v versus t')
xlabel('Values of t')
ylabel('Values of v')
grid
```

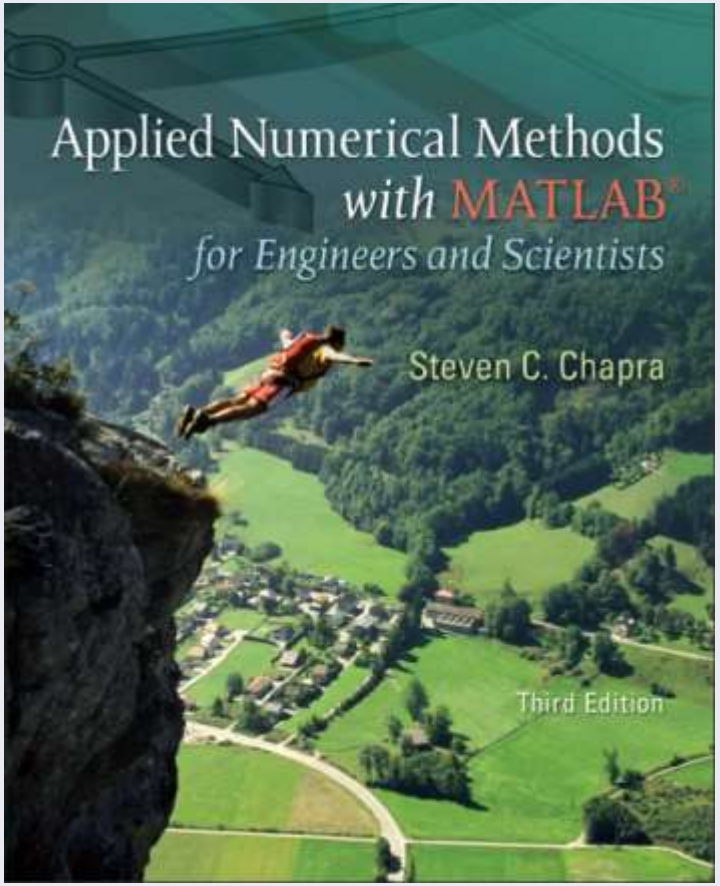


Plotting Options

- When plotting data, MATLAB can use several different colors, point styles, and line styles. These are specified at the end of the `plot` command using *plot specifiers* as found in Table 2.2.
- The default case for a single data set is to create a blue line with no points. If a line style is specified with no point style, no point will be drawn at the individual points; similarly, if a point style is specified with no point style, no line will be drawn.
- Examples of plot specifiers:
  - ‘ro:’ - red dotted line with circles at the points
  - ‘gd’ - green diamonds at the points with no line
  - ‘m--’ - magenta dashed line with no point symbols

Other Plotting Commands

- `hold on` and `hold off`
  - `hold on` tells MATLAB to keep the current data plotted and add the results of any further plot commands to the graph. This continues until the `hold off` command, which tells MATLAB to clear the graph and start over if another plotting command is given. `hold on` should be used *after* the first plot in a series is made.
- `subplot(m, n, p)`
  - `subplot` splits the figure window into an `m`x`n` array of small axes and makes the pth one active. Note - the first subplot is at the top left, then the numbering continues across the row. This is different from how elements are numbered within a matrix!



Part 1
Chapter 3

Programming with MATLAB

PowerPoints organized by Dr. Michael R. Gustafson II, Duke University and
Prof. Steve Chapra, Tufts University

All images copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

Chapter Objectives

- Learning how to create well-documented M-files in the edit window and invoke them from the command window.
- Understanding how script and function files differ.
- Understanding how to incorporate help comments in functions.
- Knowing how to set up M-files so that they interactively prompt users for information and display results in the command window.
- Understanding the role of subfunctions and how the are accessed.
- Knowing how to create and retrieve data files.

Objectives (cont)

- Learning how to write clear and well-documented M-files by employing structured programming constructs to implement logic and repetition.
- Recognizing the difference between **`if...elseif`** and **`switch`** constructs.
- Recognizing the difference between **`for...end`** and **`while`** structures.
- Understanding what is meant by vectorization and why it is beneficial.
- Knowing how to animate MATLAB plots.
- Understanding how anonymous functions can be employed to pass function functions to function M-files.

M-files

- While commands can be entered directly to the command window, MATLAB also allows you to put commands in text files called *M-files*. *M-files* are so named because the files are stored with a `.m` extension.
- There are two main kinds of M-file
  - Script files
  - Function files

Script Files

- A *script file* is merely a set of MATLAB commands that are saved on a file - when MATLAB runs a script file, it is as if you typed the characters stored in the file on the command window.
- Scripts can be executed either by typing their name (without the .m) in the command window, by selecting the `Debug, Run` (or `Save and Run`) command in the editing window, or by hitting the `F5` key while in the editing window. Note that the latter two options will save any edits you have made, while the former will run the file as it exists on the drive.

Function Files

- *Function files* serve an entirely different purpose from script files. Function files can accept input arguments from and return outputs to the command window, but variables created and manipulated within the function do not impact the command window.

Function File Syntax

- The general syntax for a function is:

```
function outvar = funcname(arglist)
% helpcomments
statements
outvar = value;
```

  where
  - *outvar*: output variable name
  - *funcname*: function's name
  - *arglist*: input argument list; comma-delimited list of what the function calls values passed to it
  - *helpcomments*: text to show with *help funcname*
  - *statements*: MATLAB commands for the function

Subfunctions

- A function file can contain a single function, but it can also contain a *primary function* and one or more *subfunctions*
- The primary function is whatever function is listed first in the M-file - its function name should be the same as the file name.
- Subfunctions are listed below the primary function. Note that they are *only* accessible by the main function and subfunctions within the same M-file and *not* by the command window or any other functions or scripts.

Input

- The easiest way to get a value from the user is the input command:
  - `n = input('promptstring')`
    MATLAB will display the characters in *promptstring*, and whatever value is typed is stored in *n*. For example, if you type *pi*, *n* will store 3.1416…
  - `n = input('promptstring', 's')`
    MATLAB will display the characters in *promptstring*, and whatever characters are typed will be stored as a string in *n*. For example, if you type *pi*, *n* will store the letters *p* and *i* in a 2x1 char array.

Output

- The easiest way to display the value of a matrix is to type its name, but that will not work in function or script files. Instead, use the `disp` command

  ```
  disp(value)
  ```

  will show the *`value`* on the screen.
- If *`value`* is a string, enclose it in single quotes.

Formatted Output

- For formatted output, or for output generated by combining variable values with literal text, use the *fprintf* command:

```
fprintf('format', x, y,...)
```

  where *format* is a string specifying how you want the value of the variables *x*, *y*, and more to be displayed - including literal text to be printed along with the values.
- The values in the variables are formatted based on format codes.

Format and Control Codes

- Within the *format* string, the following format codes define how a numerical value is displayed:
  %d - integer format
  %e - scientific format with lowercase e
  %E - scientific format with uppercase E
  %f - decidmal format
  %g - the more compact of %e or %f
- The following control codes produce special results within the *format* string:
  \n - start a new line
  \t - tab
  \\ - print the \ character
- To print a ' put a pair of ' in the *format* string

Creating and Accessing Files

- MATLAB has a built-in file format that may be used to save and load the values in variables.
- `save` *`filename var1 var2 ... varn`* saves the listed variables into a file named *`filename`*`.mat`. If no variable is listed, all variables are saved.
- `load` *`filename var1 var2 ...varn`* loads the listed variables from a file named *`filename`*`.mat`. If no variable is listed, all variables in the file are loaded.
- Note - these are not text files!

ASCII Files

- To create user-readable files, append the flag `-ascii` to the end of a `save` command. This will save the data to a text file in the same way that `disp` sends the data to a screen.
- Note that in this case, MATLAB does *not* append anything to the file name so you may want to add an extension such as *.txt* or .*dat*.
- To load a rectangular array from a text file, simply use the *load* command and the file name. The data will be stored in a matrix with the same name as the file (but without any extension).

Structured Programming

- Structured programming allows MATLAB to make decisions or selections based on conditions of the program.
- Decisions in MATLAB are based on the result of logical and relational operations and are implemented with `if`, `if…else`, and `if…elseif` structures.
- Selections in MATLAB are based on comparisons with a test expression and are implemented with `switch` structures.

Relational Operators

- From Table 3.2: Summary of relational operators in MATLAB:

Example	Operator	Relationship
`x == 0`	`==`	Equal
`unit ~= 'm'`	`~=`	Not equal
`a < 0`	`<`	Less than
`s > t`	`>`	Greater than
`3.9 <= a/3`	`<=`	Less than or equal to
`r >= 0`	`>=`	Greater than or equal to

Logical Operators

- `~x` (Not): true if `x` is false (or zero); false otherwise
- `x & y` (And): true if both `x` and `y` are true (or non-zero)
- `x | y` (Or): true if either `x` or `y` are true (or non-zero)

Order of Operations

- Priority can be set using parentheses. After that, Mathematical expressions are highest priority, followed by relational operators, followed by logical operators. All things being equal, expressions are performed from left to right.
- *Not* is the highest priority logical operator, followed by *And* and finally *Or*
- Generally, do not combine two relational operators! If `x`=5, 3<`x`<4 should be false (mathematically), but it is calculated as an expression in MATLAB as: `3<5<4`, which leads to `true<4` at which point `true` is converted to `1`, and `1<4` is true!
- Use `(3<x)&(x<4)` to properly evaluate.

Decisions

- Decisions are made in MATLAB using `if` structures, which may also include several `elseif` branches and possibly a catch-all `else` branch.
- Deciding which branch runs is based on the result of *conditions* which are either true or false.
  - If an `if` tree hits a *true* condition, that branch (and that branch only) runs, then the tree terminates.
  - If an `if` tree gets to an `else` statement without running any prior branch, that branch will run.
- *Note* - if the *condition* is a matrix, it is considered true if and only if all entries are true (or non-zero).

Selections

- Selections are made in MATLAB using switch structures, which may also include a catch-all otherwise choice.
- Deciding which branch runs is based on comparing the value in some test expression with values attached to different cases.
  - If the test expression matches the value attached to a case, that case's branch will run.
  - If no cases match and there is an otherwise statement, that branch will run.

Loops

- Another programming structure involves loops, where the same lines of code are run several times. There are two types of loop:
  - A for loop ends after a specified number of repetitions established by the number of columns given to an index variable.
  - A while loop ends on the basis of a logical condition.

`for` Loops

- One common way to use a `for…end` structure is:

```
for index = start:step:finish
   statements
end
```

where the *`index`* variable takes on successive values in the vector created using the : operator.

Vectorization

- Sometimes, it is more efficient to have MATLAB perform calculations on an entire array rather than processing an array element by element. This can be done through *vectorization*.

`for` loop	Vectorization

`i = 0;`
`for t = 0:0.02:50`
`  i = i + 1;`
`  y(i) = cos(t);`
`end`
	`t = 0:0.02:50;`
	`y = cos(t);`

`while` Loops

- A while loop is fundamentally different from a for loop since while loops can run an indeterminate number of times. The general syntax is

```
while condition
   statements
end
```

  where the *`condition`* is a logical expression. If the *`condition`* is true, the *`statements`* will run and when that is finished, the loop will again check on the *`condition`*.
- Note - though the *`condition`* may become false as the *`statements`* are running, the only time it matters is after all the statements have run.

Early Termination

- Sometimes it will be useful to break out of a for or while loop early - this can be done using a break statement, generally in conjunction with an if structure.
- Example:

```
x = 24
while (1)
  x = x - 5
  if x < 0, break, end
end
```

will produce x values of 24, 19, 14, 9, 4, and -1, then stop.

Animation

- Two ways to animate plots in MATLAB:
  - Using looping with simple plotting functions
    - This approach merely replots the graph over and over again.
    - Important to use the `axis` command so that the plots scales are fixed.
  - Using special function: `getframe` and `movie`
    - This allows you to capture a sequence of plots (`getframe`) and then play them back (`movie`).

Example

- The (x, y) coordinates of a projectile can be generated as a function of time, t,with the following parametric equations

$$x = v_0 \cos(\theta_0 t)$$

$$y = v_0 \sin(\theta_0 t) - 0.5\, g t^2$$

where v_0 = initial velocity (m/s)

θ_0 = initial angle (radians)

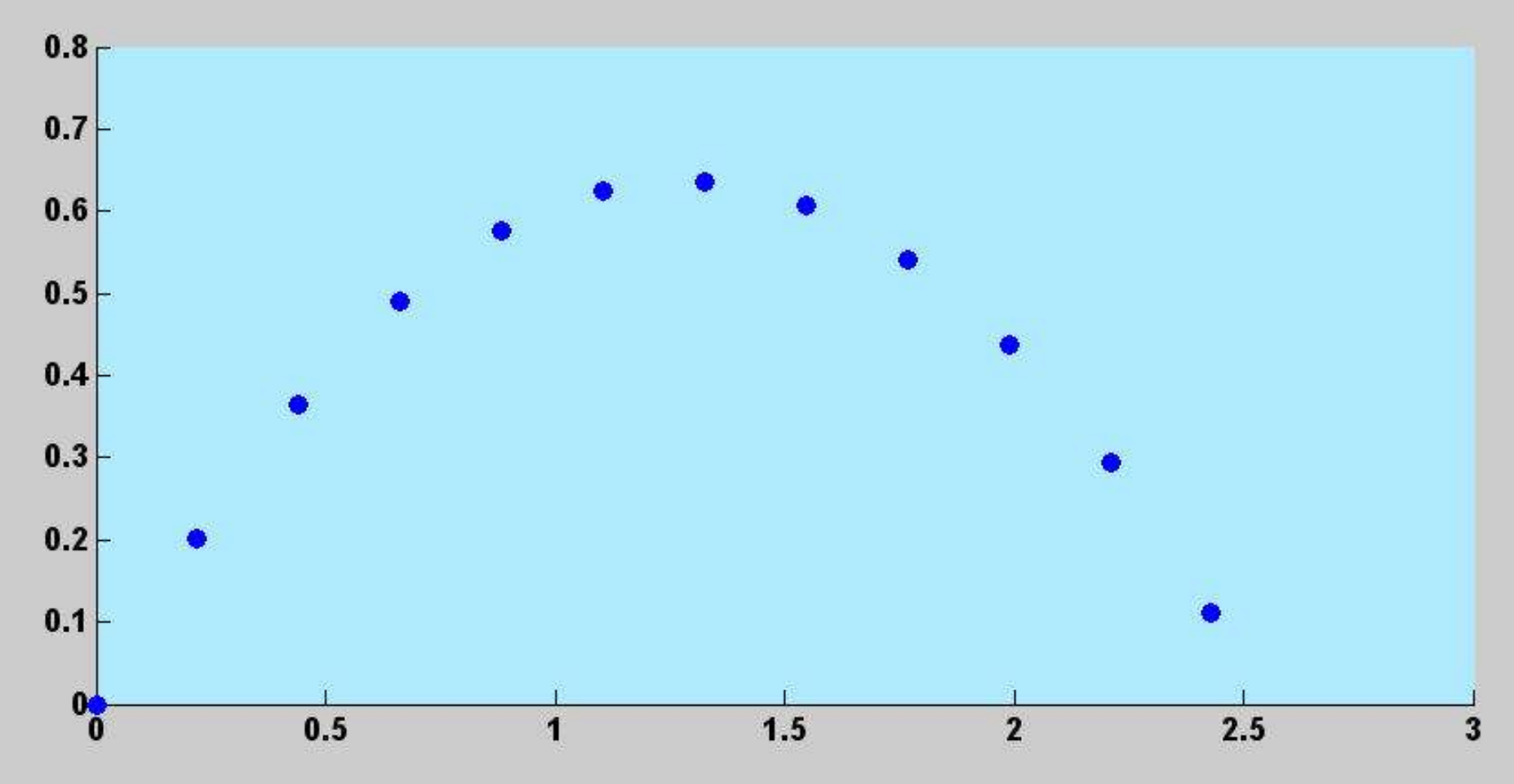
g = gravitational constant (= 9.81 m/s^2)

Script

- The following code illustrates both approaches:

```
clc,clf,clear
g=9.81; theta0=45*pi/180; v0=5;
t(1)=0;x=0;y=0;
plot(x,y,'o','MarkerFaceColor','b','MarkerSize',8)
axis([0 3 0 0.8])
M(1)=getframe;
dt=1/128;
for j = 2:1000
  t(j)=t(j-1)+dt;
  x=v0*cos(theta0)*t(j);
  y=v0*sin(theta0)*t(j)-0.5*g*t(j)^2;
  plot(x,y,'o','MarkerFaceColor','b','MarkerSize',8)
  axis([0 3 0 0.8])
  M(j)=getframe;
  if y<=0, break, end
end
pause
movie(M,1)
```

Result



Nesting and Indentation

- Structures can be placed within other structures. For example, the `statements` portion of a `for` loop can be comprised of an `if...elseif...else` structure.
- For clarity of reading, the `statements` of a structure are generally indented to show which lines of controlled are under the control of which structure.

Anonymous & Inline Functions

- *Anonymous functions* are simple one-line functions created without the need for an M-file
  ```
  fhandle = @(arg1, arg2, ...) expression
  ```
- *Inline functions* are essentially the same as anonymous functions, but with a different syntax:
  ```
  fhandle = inline('expression', 'arg1', 'arg2',...)
  ```
- Anonymous functions can access the values of variables in the workspace upon creation, while inlines cannot.

Function Functions

- *Function functions* are functions that operate on other functions which are passed to it as input arguments. The input argument may be the handle of an anonymous or inline function, the name of a built-in function, or the name of a M-file function.
- Using function functions will allow for more dynamic programming.

MATH 259
Numerical Methods
Introduction to MATLAB
Part 2

Original Slides by
Gerald W. Recktenwald

These slides are a supplement to the book *Numerical Methods with* MATLAB*: Implementations and Applications*, by Gerald W. Recktenwald, © 2001, Prentice-Hall, Upper Saddle River, NJ. These slides are © 2001 Gerald W. Recktenwald. The PDF version of these slides may be downloaded or stored or printed only for noncommercial, educational use. The repackaging or sale of these slides in any form, without written consent of the author, is prohibited.

The latest version of this PDF file, along with other supplemental material for the book, can be found at `www.prenhall.com/recktenwald`.

Version 0.97 August 28, 2001

Overview

- Script m-files
  - ▷ Creating
  - ▷ Side effects
- Function m-files
  - ▷ Syntax of I/O parameters
  - ▷ Text output
  - ▷ Primary and secondary functions
- Flow control
  - ▷ Relational operators
  - ▷ Conditional execution of blocks
  - ▷ Loops
- Vectorization
  - ▷ Using vector operations instead of loops
  - ▷ Preallocation of vectors and matrices
  - ▷ Logical and array indexing
- Programming tricks
  - ▷ Variable number of I/O parameters
  - ▷ Indirect function evaluation
  - ▷ Inline function objects
  - ▷ Global variables

Preliminaries

- Programs are contained in m-files
  - ▷ Plain text files – not binary files produced by word processors
  - ▷ File must have ".m" extension
- m-file must be in the path
  - ▷ MATLAB maintains its own internal path
  - ▷ The path is the list of directories that MATLAB will search when looking for an m-file to execute.
  - ▷ A program can exist, and be free of errors, but it will not run if MATLAB cannot find it.
  - ▷ Manually modify the path with the **path**, **addpath**, and **rmpath** built-in functions.
  - ▷ . . . or use interactive Path Browser

Script Files

- Not really programs
  - ▷ No input/output parameters
  - ▷ Script variables are part of workspace
- Useful for tasks that never change
- Useful as a tool for documenting homework:
  - ▷ Write a *function* that solves the problem for *arbitrary* parameters
  - ▷ Use a `script` to run function for specific parameters required by the assignment

Free Advice: Scripts offer no advantage over functions. Functions have many advantages over scripts. Always use functions instead of scripts.

Script to Plot $\tan(\theta)$ (1)

Enter statements in file called `tanplot.m`

1. Choose **New** -> **Script** from **File** menu
2. Enter lines listed below

Contents of `tanplot.m`:

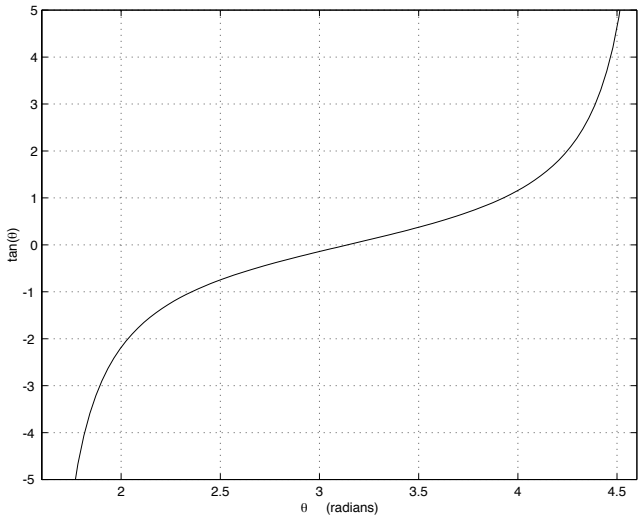
```
theta = linspace(1.6,4.6);
tandata = tan(theta);
plot(theta,tandata);
xlabel('\theta     (radians)');
ylabel('tan(\theta)');
grid on;
axis([min(theta) max(theta) -5 5]);
```

3. Choose **Save As** from **File** menu
   Save as `tanplot.m`
4. Run it

```
>> tanplot
```

Script to Plot $\tan(\theta)$ (2)

Running `tanplot` produces the following plot:



If the plot needs to be changed, edit the `tanplot` script and rerun it. This saves the effort of typing in the commands. The `tanplot` script also provides written documentation of how to create the plot.

Example: Put a % character at beginning of the line containing the `axis` command, then rerun the script

Script Side-Effects (1)

All variables created in a script file are added to the workplace. This may have undesirable effects because

- Variables already existing in the workspace may be overwritten
- The execution of the script can be affected by the state variables in the workspace.

Example: *The* `easyplot` *script*

```
% easyplot:  Script to plot data in file xy.dat

%  Load the data
D = load('xy.dat');       %  D is a matrix with two columns
x = D(:,1);  y = D(:,2);  %  x in 1st column, y in 2nd column

plot(x,y)                 %  Generate the plot and label it
xlabel('x axis, unknown units')
ylabel('y axis, unknown units')
title('Plot of generic x-y data set')
```

Script Side-Effects (2)

The `easyplot` script affects the workspace by creating three variables:

```
>> clear
>> who
```

(no variables show)

```
>> easyplot
>> who

Your variables are:

D         x         y
```

The `D`, `x`, and `y` variables are left in the workspace. These generic variable names might be used in another sequence of calculations in the same MATLAB session.

Function m-files (1)

- Functions are subprograms:
  - ▷ Functions use input and output parameters to communicate with other functions and the command window
  - ▷ Functions use *local* variables that exist only while the function is executing. Local variables are distinct from variables of the same name in the workspace or in other functions.
- Input parameters allow the same calculation procedure (same algorithm) to be applied to different data. Thus, function m-files are *reusable*.
- Functions can call other functions.
- Specific tasks can be encapsulated into functions. This modular approach enables development of structured solutions to complex problems.

Function m-files (2)

Syntax:

The first line of a function m-file has the form:

```
function [outArgs] = funName(inArgs)
```

outArgs are enclosed in `[ ]`

- *outArgs* is a comma-separated list of variable names
- `[ ]` is optional if there is only one parameter
- functions with no *outArgs* are legal

inArgs are enclosed in `( )`

- *inArgs* is a comma-separated list of variable names
- functions with no *inArgs* are legal

Function Input and Output (1)

Examples: *Demonstrate use of I/O arguments*

- `twosum.m` — two inputs, no output
- `threesum.m` — three inputs, one output
- `addmult.m` — two inputs, two outputs

Function Input and Output (2)

twosum.m

```
function twosum(x,y)
% twosum  Add two matrices
%         and print the result
x+y
```

threesum.m

```
function s = threesum(x,y,z)
% threesum  Add three variables
%           and return the result
s = x+y+z;
```

addmult.m

```
function [s,p] = addmult(x,y)
% addmult  Compute sum and product
%          of two matrices
s = x+y;
p = x*y;
```

Function Input and Output Examples (3)

Example: *Experiments with* `twosum`*:*

```
>> twosum(2,2)
ans =
      4

>> x = [1 2];  y = [3 4];
>> twosum(x,y)
ans =
     4     6

>> A = [1 2; 3 4];  B = [5 6; 7 8];
>> twosum(A,B);
ans =
     6     8
    10    12

>> twosum('one','two')
ans =
   227   229   212
```

Note: The result of the addition inside `twosum` is exposed because the `x+y` expression does not end in a semicolon.

Function Input and Output Examples (4)

Example: *Experiments with* `twosum`*:*

```
>> clear
>> x = 4; y = -2;
>> twosum(1,2)
ans =
     3

>> x+y
ans =
     2

>> disp([x y])
     4    -2

>> who

Your variables are:

ans       x         y
```

In this example, the `x` and `y` variables defined in the workspace are distinct from the `x` and `y` variables defined in `twosum`. The `x` and `y` in `twosum` are *local* to `twosum`.

Function Input and Output Examples (5)

Example: *Experiments with* `threesum`*:*

```
>> a = threesum(1,2,3)
a =
    6

>> threesum(4,5,6)
ans =
   15

>> b = threesum(7,8,9);
```

Note: The last statement produces no output because the assignment expression ends with a semicolon. The value of 24 is stored in `b`.

Function Input and Output Examples (6)

Example: *Experiments with* `addmult`*:*

```
>> [a,b] = addmult(3,2)
a =
    5
b =
    6

>> addmult(3,2)
ans =
    5

>> v = addmult(3,2)
v =
    5
```

Note: `addmult` *requires* two return variables. Calling `addmult` with no return variables or with one return variable causes undesired behavior.

Summary of Input and Output Parameters

- Values are communicated through input arguments and output arguments.
- Variables defined inside a function are *local* to that function. Local variables are invisible to other functions and to the command environment.
- The number of return variables should match the number of output variables provided by the function.

Text Input and Output

It is usually desirable to print results to the screen or to a file. On rare occasions it may be helpful to prompt the user for information not already provided by the input parameters to a function.

Inputs to functions:

- **input** function can be used.
- Input parameters to functions are preferred.

Text output from functions:

- **disp** function for simple output
- **fprintf** function for formatted output.

Prompting for User Input

The **input** function can be used to prompt the user for numeric or string input.

```
>> x = input('Enter a value for x');

>> yourName = input('Enter your name','s');
```

Prompting for input betrays the MATLAB novice. It is a nuisance to competent users, and makes automation of computing tasks impossible.

Free Advice: Avoid using the **input** function. Rarely is it necessary. All inputs to a function should be provided via the input parameter list.

Text Output with disp and fprintf

Output to the command window is achieved with either the **disp** function or the **fprintf** function. Output to a file requires the **fprintf** function.

disp	Simple to use. Provides limited control over appearance of output.
fprintf	Slightly more complicated than `disp`. Provides total control over appearance of output.

The disp function (1)

Syntax:

```
disp(outMatrix)
```

where *outMatrix* is *either* a string matrix or a numeric matrix.

Examples: *Numeric output*

```
>> disp(5)
       5

>> x = 1:3;  disp(x)
       1     2     3

>> y = 3-x;  disp([x; y])
       1     2     3
       2     1     0

>> disp([x y])
       1     2     3     2     1     0

>> disp([x' y])
???  All matrices on a row in the bracketed expression
 must have the same number of rows.
```

Note: The last statement shows that the input to **disp** must be a legal matrix.

The disp function (2)

Examples: *String output*

```
>> disp('Hello, world!)
Hello, world!

>> s = 'MATLAB 6 is built with LAPACK';  disp(s)
MATLAB 6 is built with LAPACK

>> t = 'Earlier versions used LINPACK and EISPACK';
>> disp([s; t])
???  All rows in the bracketed expression
 must have the same number of columns.

>> disp(char(s,t))
MATLAB 6 is built with LAPACK
Earlier versions used LINPACK and EISPACK
```

The `disp[s; t]` expression causes an error because `s` has fewer elements than `t`. The built-in **char** function constructs a string matrix by putting each input on a separate row and padding the rows with blanks as necessary.

```
>> S = char(s,t);
>> length(s),  length(t),  length(S(1,:))
ans =
    29
ans =
    41
ans =
    41
```

The num2str function (1)

The **num2str** function is often used to with the **disp** function to create a labeled output of a numeric value.

Syntax:

stringValue = num2str(*numericValue*)

converts *numericValue* to a string representation of that numeric value.

Examples:

```
>> num2str(pi)
ans =
3.1416

>> A = eye(3)
A =
     1     0     0
     0     1     0
     0     0     1

>> S = num2str(A)
S =
1  0  0
0  1  0
0  0  1
```

The num2str function (2)

Although A and S appear to contain the same values, they are not equivalent. A is a numeric matrix, and S is a string matrix.

```
>> clear
>> A = eye(3);  S = num2str(A);  B = str2num(S);
>> A-S
??? Error using ==> -
Matrix dimensions must agree.

>> A-B
ans =
     0     0     0
     0     0     0
     0     0     0

>> whos
  Name      Size         Bytes  Class

  A         3x3             72  double array
  B         3x3             72  double array
  S         3x7             42  char array
  ans       3x3             72  double array

Grand total is 48 elements using 258 bytes
```

Using num2str with disp (1)

Combine **num2str** and **disp** to print a labeled output of a numeric value

```
>> x = sqrt(2);
>> outString = ['x = ',num2str(x)];
>> disp(outString)
x = 1.4142
```

or, build the input to `disp` on the fly

```
>> disp(['x = ',num2str(x)]);
x = 1.4142
```

Using num2str with disp (2)

The

```
disp(['x = ',num2str(x)]);
```

construct works when `x` is a row vector, but not when `x` is a column vector or matrix

```
>> z = y';
>> disp(['z = ',num2str(z)])
???  All matrices on a row in the bracketed expression
 must have the same number of rows.
```

Instead, use two `disp` statements to display column of vectors or matrices

```
>> disp('z = ');  disp(z)
z =
     1
     2
     3
     4
```

Using num2str with disp (3)

The same effect is obtained by simply entering the name of the variable with no semicolon at the end of the line.

```
>> z                    (enter z and press return)
z =
     1
     2
     3
     4
```

The format function

The **format** function controls the precision of **disp** output.

```
>> format short
>> disp(pi)
      3.1416

>> format long
>> disp(pi)
      3.14159265358979
```

Alternatively, a second parameter can be used to control the precision of the output of **num2str**

```
>> disp(['pi = ',num2str(pi,2)])
pi = 3.1

>> disp(['pi = ',num2str(pi,4)])
pi = 3.142

>> disp(['pi = ',num2str(pi,8)])
pi = 3.1415927
```

The fprintf function (1)

Syntax:

```
fprintf(outFormat,outVariables)
fprintf(fileHandle,outFormat,outVariables)
```

uses the *outFormat* string to convert *outVariables* to strings that are printed. In the first form (no *fileHandle*) the output is displayed in the command window. In the second form, the output is written to a file referred to by the *fileHandle* (more on this later).

Notes to C programmers:

1. The MATLAB **fprintf** function uses single quotes to define the format string.
2. The **fprintf** function is vectorized. (See examples below.)

Example:

```
>> x = 3;
>> fprintf('Square root of %g is %8.6f\n',x,sqrt(x));

The square root of 3 is 1.732051
```

The fprintf function (2)

The *outFormat* string specifies how the *outVariables* are converted and displayed. The *outFormat* string can contain any text characters. It also must contain a *conversion code* for each of the *outVariables*. The following table shows the basic conversion codes.

Code	Conversion instruction
%s	format as a string
%d	format with no fractional part (integer format)
%f	format as a floating-point value
%e	format as a floating-point value in scientific notation
%g	format in the most compact form of either %f or %e
\n	insert newline in output string
\t	insert tab in output string

The fprintf function (3)

In addition to specifying the type of conversion (e.g. %d, %f, %e) one can also specify the width and precision of the result of the conversion.

Syntax:

%*w*d
%*w*.*p*f
%*w*.*p*e

where *w* is the number of characters in the width of the final result, and *p* is the number of digits to the right of the decimal point to be displayed.

Examples:

Format String	Meaning
%14.5f	use floating point format to convert a numerical value to a string 14 characters wide with 5 digits after the decimal point
%12.3e	use scientific notation format to convert numerical value to a string 12 characters wide with 3 digits after the decimal point. The 12 characters for the string include the e+00 or e−00 (or e+000 or e−000 on Windows™)

The fprintf function (4)

More examples of conversion codes

Value	%8.4f	%12.3e	%10g	%8d
2	2.0000	2.000e+00	2	2
sqrt(2)	1.4142	1.414e+00	1.41421	1.414214e+00
sqrt(2e-11)	0.0000	4.472e-06	4.47214e-06	4.472136e-06
sqrt(2e11)	447213.5955	4.472e+05	447214	4.472136e+05

The fprintf function (5)

The **fprintf** function is vectorized. This enables printing of vectors and matrices with compact expressions. It can also lead to some undesired results.

Examples:

```
>> x = 1:4;  y = sqrt(x);
>> fprintf('%9.4f\n',y)
   1.0000
   1.4142
   1.7321
   2.0000
```

The `%9.4f` format string is reused for each element of `y`. The recycling of a format string may not always give the intended result.

```
>> x = 1:4;  y = sqrt(x);
>> fprintf('y = %9.4f\n',y)
y =    1.0000
y =    1.4142
y =    1.7321
y =    2.0000
```

The fprintf function (6)

Vectorized **fprintf** cycles through the *outVariables by columns*. This can also lead to unintended results

```
>> A = [1 2 3; 4 5 6; 7 8 9]
A =
     1     2     3
     4     5     6
     7     8     9

>> fprintf('%8.2f  %8.2f  %8.2f\n',A)
    1.00      4.00      7.00
    2.00      5.00      8.00
    3.00      6.00      9.00
```

How to print a table with fprintf (1)

Many times a tabular display of results is desired.

The `boxSizeTable` function listed below, shows how the **fprintf** function creates column labels and formats numeric data into a tidy tabular display. The **for** loop construct is discussed later in these slides.

```
function boxSizeTable
% boxSizeTable  Demonstrate tabular output with fprintf

% --- labels and sizes for shiping containers
label = char('small','medium','large','jumbo');
width = [5; 5; 10; 15];
height = [5; 8; 15; 25];
depth = [15; 15; 20; 35];
vol = width.*height.*depth/10000;   %  volume in cubic meters

fprintf('\nSizes of boxes used by ACME  Delivery Service\n\n');
fprintf('size          width     height    depth    volume\n');
fprintf('               (cm)      (cm)      (cm)     (m^3)\n');
for i=1:length(width)
  fprintf('%-8s  %8d  %8d  %8d   %9.5f\n',...
          label(i,:),width(i),height(i),depth(i),vol(i))
end
```

Note: **length** is a built-in function that returns the number of elements in a vector. `width`, `height`, and `depth` are local variables in the `boxSizeTable` function.

How to print a table with fprintf (2)

Example: *Running* `boxSizeTable` *gives*

```
>> boxSizeTable

Sizes of boxes used by ACME Delivery Service

size          width     height    depth     volume
               (cm)      (cm)      (cm)      (m^3)
small            5         5        15     0.03750
medium           5         8        15     0.06000
large           10        15        20     0.30000
jumbo           15        25        35     1.31250
```

The fprintf function (3)

File Output with **fprintf** requires creating a *file handle* with the **fopen** function. All aspects of formatting and vectorization discussed for screen output still apply.

Example: *Writing contents of a vector to a file.*

```
x = ...                                % content of x
fout = fopen('myfile.dat','wt');       % open myfile.dat
fprintf(fout,'   k     x(k)\n');
for k=1:length(x)
   fprintf(fout,'%4d     %5.2f\n',k,x(k));
end
fclose(fout)                             % close myfile.dat
```

Flow Control (1)

To enable the implementation of computer algorithms, a computer language needs control structures for

- Repetition: looping or iteration
- Conditional execution: branching
- Comparison

We will consider these in reverse order.

Comparison
Comparison is achieved with *relational operators*. Relational operators are used to test whether two values are equal, or whether one value is greater than or less than another. The result of a comparison may also be modified by *logical operators*.

Relational Operators (1)

Relational operators are used in comparing two values.

Operator	Meaning
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
~=	not equal to

The result of applying a relational operator is a logical value, i.e. the result is either *true* or *false*.

In MATLAB any nonzero value, including a non-empty string, is equivalent to *true*. Only zero is equivalent to *false*.

Note: The <=, >=, and ~= operators have "=" as the *second* character. =<, => and =~ are not valid operators.

Relational Operators (2)

The result of a relational operation is a true or false value.

Examples:

```
>> a = 2;  b = 4;
>> aIsSmaller = a < b
aIsSmaller =
     1

>> bIsSmaller = b < a
bIsSmaller =
     0
```

Relational operations can also be performed on matrices of the same shape, e.g.,

```
>> x = 1:5;   y = 5:-1:1;
>> z = x>y
z =
     0     0     0     1     1
```

Logical Operators

Logical operators are used to combine logical expressions (with "and" or "or"), or to change a logical value with "not"

Operator	Meaning
&	and
|	or
~	not

Examples:

```
>> a = 2;  b = 4;
>> aIsSmaller = a < b;
>> bIsSmaller = b < a;
>> bothTrue = aIsSmaller & bIsSmaller
bothTrue =
     0

>> eitherTrue = aIsSmaller | bIsSmaller
eitherTrue =
     1

>> ~eitherTrue
ans =
     0
```

Logical and Relational Operators

Summary:

- Relational operators involve comparison of two values.
- The result of a relational operation is a logical (True/False) value.
- Logical operators combine (or negate) logical values to produce another logical value.
- There is always more than one way to express the same comparison

Conditional Execution

Conditional Execution or Branching:

As the result of a comparison, or another logical (true/false) test, selected blocks of program code are executed or skipped.

Conditional execution is implemented with `if`, `if...else`, and `if...elseif` constructs, or with a `switch` construct.

There are three types of `if` constructs

1. Plain `if`
2. `if...else`
3. `if...elseif`

if Constructs

Syntax:

```
if expression
    block of statements
end
```

The *block of statements* is executed only if the *expression* is true.

Example:

```
if a < 0
   disp('a is negative');
end
```

One line format uses comma after `if` *expression*

```
if a < 0,  disp('a is negative'); end
```

if. . . else

Multiple choices are allowed with **if. . . else** and **if. . . elseif** constructs

```
if x < 0
   error('x is negative; sqrt(x) is imaginary');
else
   r = sqrt(x);
end
```

if. . . elseif

It's a good idea to include a default **else** to catch cases that don't match preceding **if** and **elseif** blocks

```
if x > 0
   disp('x is positive');
elseif x < 0
   disp('x is negative');
else
   disp('x is exactly zero');
end
```

The switch Construct

A switch construct is useful when a test value can take on discrete values that are either integers or strings.

Syntax:

```
switch  expression
  case value1,
    block of statements
  case value2,
    block of statements
    ⋮
  otherwise,
    block of statements
end
```

Example:

```
color = '...';    %  color is a string
switch color
    case 'red'
        disp('Color is red');
    case 'blue'
        disp('Color is blue');
    case 'green'
        disp('Color is green');
    otherwise
        disp('Color is not red, blue, or green');
end
```

Flow Control (3)

Repetition or Looping

A sequence of calculations is repeated until *either*

1. All elements in a vector or matrix have been processed

or

2. The calculations have produced a result that meets a predetermined termination criterion

Looping is achieved with `for` loops and `while` loops.

for loops

`for` loops are most often used when each element in a vector or matrix is to be processed.

Syntax:

```
for index = expression
    block of statements
end
```

Example: *Sum of elements in a vector*

```
x = 1:5;              %  create a row vector
sumx = 0;             %  initialize the sum
for k = 1:length(x)
   sumx = sumx + x(k);
end
```

for loop variations

Example: *A loop with an index incremented by two*

```
for k = 1:2:n
   ...
end
```

Example: *A loop with an index that counts down*

```
for k = n:-1:1
   ...
end
```

Example: *A loop with non-integer increments*

```
for x = 0:pi/15:pi
   fprintf('%8.2f  %8.5f\n',x,sin(x));
end
```

Note: In the last example, `x` is a *scalar* inside the loop. Each time through the loop, `x` is set equal to one of the *columns* of `0:pi/15:pi`.

while loops (1)

`while` loops are most often used when an iteration is repeated until some termination criterion is met.

Syntax:

```
while expression
   block of statements
end
```

The *block of statements* is executed as long as *expression* is true.

Example: *Newton's method for evaluating* $\sqrt{x}$

$$r_k = \frac{1}{2}\left(r_{k-1} + \frac{x}{r_{k-1}}\right)$$

```
r = ...                % initialize
rold = ...
while abs(rold-r) > delta
   rold = r;
   r = 0.5*(rold + x/rold);
end
```

while loops (2)

It is (almost) always a good idea to put a limit on the number of iterations to be performed by a `while` loop.

An improvement on the preceding loop,

```
maxit = 25;
it = 0;
while abs(rold-r) > delta & it<maxit
   rold = r;
   r = 0.5*(rold + x/rold);
   it = it + 1;
end
```

while loops (3)

The **break** and **return** statements provide an alternative way to exit from a loop construct. **break** and **return** may be applied to `for` loops or `while` loops.

break is used to escape from an enclosing `while` or `for` loop. Execution continues at the end of the enclosing loop construct.

return is used to force an exit from a **function**. This can have the effect of escaping from a loop. Any statements following the loop that are in the function body are skipped.

The break command

Example: *Escape from a* `while` *loop*

```
function k = breakDemo(n)
% breakDemo  Show how the "break" command causes
%            exit from a while loop.
%            Search a random vector to find index
%            of first element greater than 0.8.
%
% Synopsis:  k = breakDemo(n)
%
% Input:     n = size of random vector to be generated
%
% Output:    k = first (smallest) index in x such that x(k)>0.8
x = rand(1,n);
k = 1;
while k<=n
   if x(k)>0.8
     break
   end
   k = k + 1;
end
fprintf('x(k)=%f   for k = %d   n = %d\n',x(k),k,n);

%  What happens if loop terminates without finding x(k)>0.8 ?
```

The return command

Example: *Return from within the body of a function*

```
function k = returnDemo(n)
% returnDemo  Show how the "return" command
%             causes exit from a function.
%             Search a random vector to find
%             index of first element greater than 0.8.
%
% Synopsis:  k = returnDemo(n)
%
% Input:     n = size of random vector to be generated
%
% Output:    k = first (smallest) index in x
%                such that x(k)>0.8
x = rand(1,n);
k = 1;

while k<=n
   if x(k)>0.8
     return
   end
   k = k + 1;
end

%  What happens if loop terminates without finding x(k)>0.8 ?
```

Comparison of break and return

break is used to escape the current `while` or `for` loop.

return is used to escape the current function.

```
function k = demoBreak(n)

...

while k<=n
   if x(k)>0.8
     break;
   end
   k = k + 1;
end
```

jump to end of enclosing "`while ... end`" block

```
function k = demoReturn(n)

...

while k<=n
   if x(k)>0.8
     return;
   end
   k = k + 1;
end
```

return to calling function

Vectorization

Vectorization is the use of vector operations (MATLAB expressions) to process all elements of a vector or matrix. Properly vectorized expressions are equivalent to looping over the elements of the vectors or matrices being operated upon. A vectorized expression is more compact and results in code that executes faster than a non-vectorized expression.

To write vectorized code:

- Use vector operations instead of loops, where applicable
- Pre-allocate memory for vectors and matrices
- Use vectorized indexing and logical functions

Non-vectorized code is sometimes called "*scalar code*" because the operations are performed on scalar elements of a vector or matrix instead of the vector as a whole.

Free Advice: Code that is slow and correct is always better than code that is fast and incorrect. **Start with scalar code, then vectorize as needed.**

Replace Loops with Vector Operations

Scalar Code

```
for k=1:length(x)
   y(k) = sin(x(k))
end
```

Vectorized equivalent

```
y = sin(x)
```

Preallocate Memory

The following loop increases the size of s on each pass.

```
y = ...      %  some computation to define y
for j=1:length(y)
   if y(j)>0
     s(j) = sqrt(y(j));
   else
     s(j) = 0;
   end
end
```

Preallocate s before assigning values to elements.

```
y = ...      %  some computation to define y
s = zeros(size(y));
for j=1:length(y)
   if y(j)>0
     s(j) = sqrt(y(j));
   end
end
```

Vectorized Indexing and Logical Functions (1)

Thorough vectorization of code requires use of **array indexing** and **logical indexing**.

Array Indexing:

Use a vector or matrix as the "subscript" of another matrix:

```
>> x = sqrt(0:4:20)
x =
    0    2.0000    2.8284    3.4641    4.0000    4.47210

>> i = [1 2 5];
>> y = x(i)
y =
     0     2     4
```

The `x(i)` expression selects the elements of `x` having the indices in `i`. The expression `y = x(i)` is equivalent to

```
k = 0;
for i = [1 2 5]
  k = k + 1;
  y(k) = x(i);
end
```

Vectorized Indexing and Logical Functions (2)

Logical Indexing:

Use a vector or matrix as the mask to select elements from another matrix:

```
>> x = sqrt(0:4:20)
x =
     0    2.0000    2.8284    3.4641    4.0000    4.47210

>> j = find(rem(x,2)==0)
j =
     1     2     5

>> z = x(j)
z =
     0     2     4
```

The `j` vector contains the indices in `x` that correspond to elements in `x` that are integers.

Vectorized Indexing and Logical Functions (3)

Example: *Vectorization of Scalar Code*

We just showed how to pre-allocate memory in the code snippet:

```
y = ...      %  some computation to define y
s = zeros(size(y));
for j=1:length(y)
   if y(j)>0
     s(j) = sqrt(y(j));
   end
end
```

In fact, the loop can be replaced entirely by using logical and array indexing

```
y = ...                %  some computation to define y
s = zeros(size(y));
i = find(y>0);         %  indices such that y(i)>0
s(y>0) = sqrt(y(y>0))
```

If we don't mind redundant computation, the preceding expressions can be further contracted:

```
y = ...      %  some computation to define y
s = zeros(size(y));
s(y>0) = sqrt(y(y>0))
```

Vectorized Copy Operations (1)

Example: *Copy entire columns (or rows)*

Scalar Code

```
[m,n] = size(A);  %  assume A and B have
                  %  same number of rows
for i=1:m
    B(i,1) = A(i,1);
end
```

Vectorized Code

```
B(:,1) = A(:,1);
```

Vectorized Copy Operations (2)

Example: *Copy and transform submatrices*

Scalar Code

```
for j=2:3
   B(1,j) = A(j,3);
end
```

Vectorized Code

```
B(1,2:3) = A(2:3,3)'
```

Deus ex Machina

MATLAB has features to solve some recurring programming problems:

- Variable number of I/O parameters
- Indirect function evaluation with **feval**
- In-line function objects (MATLAB version 5.x)
- Global Variables

Variable Input and Output Arguments (1)

Each function has internal variables, `nargin` and `nargout`.

Use the value of `nargin` at the beginning of a function to find out how many input arguments were supplied.

Use the value of `nargout` at the end of a function to find out how many input arguments are expected.

Usefulness:

- Allows a single function to perform multiple related tasks.
- Allows functions to assume default values for some inputs, thereby simplifying the use of the function for some tasks.

Variable Input and Output Arguments (2)

Consider the built-in **plot** function

	Inside the `plot` function	
	`nargin`	`nargout`
`plot(x,y)`	2	0
`plot(x,y,'s')`	3	0
`plot(x,y,'s--')`	3	0
`plot(x1,y1,'s',x2,y2,'o')`	6	0
`h = plot(x,y)`	2	1

The values of `nargin` and `nargout` are determined when the `plot` function is invoked.

Indirect Function Evaluation (1)

The **feval** function allows a function to be evaluated indirectly.

Usefulness:

- Allows routines to be written to process an arbitrary $f(x)$.
- Separates the reusable algorithm from the problem-specific code.

Indirect Function Evaluation (2)

```
>> fsum('sin',0,pi,5)
ans =
    2.4142

>> fsum('cos',0,pi,5)
ans =
    0
```

Use of feval

```
function s = fsum(fun,a,b,n)
%  FSUM  Computes the sum of function values, f(x), at n equally
%        distributed points in an interval a <= x <= b
%
%  Synopsis:   s = fsum(fun,a,b,n)
%
%  Input:  fun = (string) name of the function to be evaluated
%          a,b = endpoints of the interval
%          n   = number of points in the interval

x = linspace(a,b,n);     %  create points in the interval
y = feval(fun,x);        %  evaluate function at sample points
s = sum(y);              %  compute the sum
```

```
function y = sincos(x)
%  SINCOS   Evaluates sin(x)*cos(x) for any input x
%
%  Synopsis:  y = sincos(x)
%
%  Input:     x = angle in radians, or vector of angles in radians
%
%  Output:    y = value of product sin(x)*cos(x) for each element in x

y = sin(x).*cos(x);
```

Inline Function Objects

MATLAB version 5.x introduced object-oriented programming extensions. Though OOP is an advanced and somewhat subtle way of programming, in-line function objects are simple to use and offer great program flexibility.

Instead of

```
function y = myFun(x)
y = x.^2 - log(x);
```

Use

```
myFun = inline( 'x.^2 - log(x)' );
```

Both definitions of `myFun` allow expressions like

```
z = myFun(3);

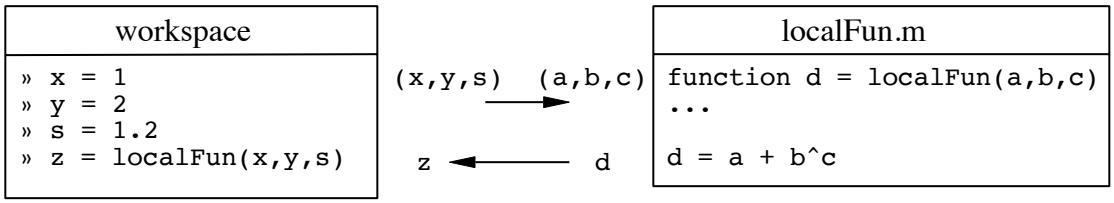
s = linspace(1,5);
t = myFun(s);
```

Usefulness:

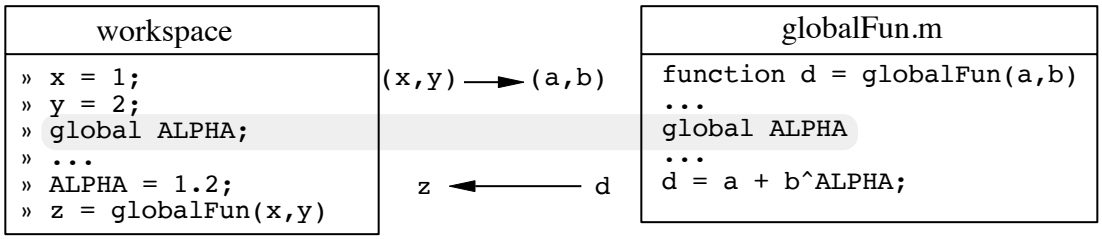
- Eliminates need to write separate m-files for functions that evaluate a simple formula.
- Useful in all situations where **feval** is used.

Global Variables

Communication of values via input and output variables



Communication of values via input and output variables
and global variables shared by the workspace and function



Usefulness:

- Allows bypassing of input parameters if no other mechanism (such as pass-through parameters) is available.
- Provides a mechanism for maintaining program state (GUI applications)